

CSE470: Software Engineering

Factory Pattern

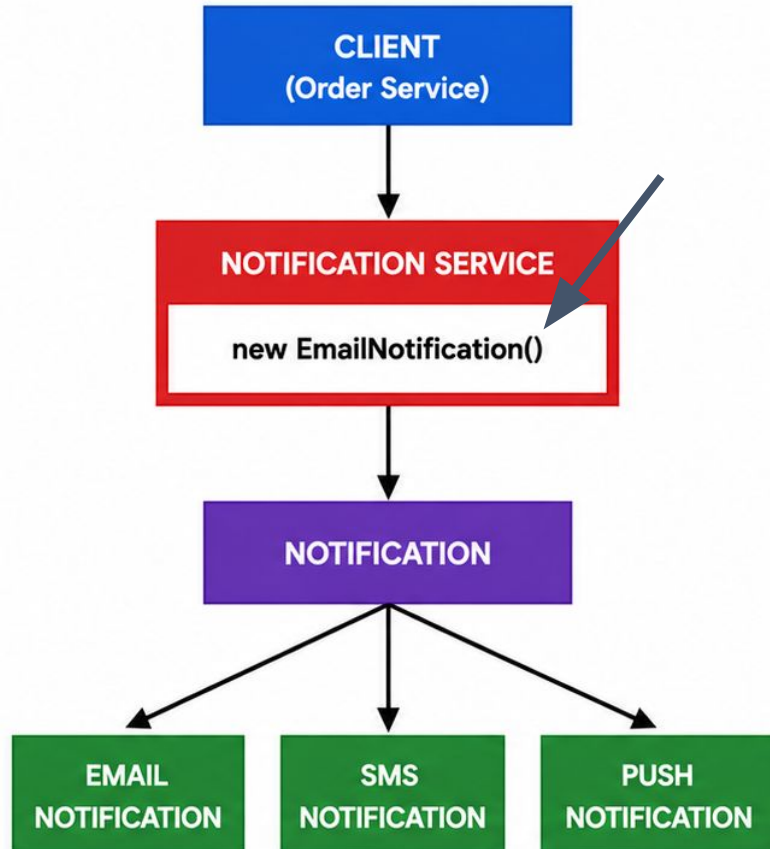


Factory Method Pattern

- ***Defines an interface for creating an object, but lets subclasses decide which class to instantiate.***
- ***Factory pattern is used to create object without exposing the instantiation logic to the client.***
- Factory Method lets a class defer instantiation to its subclasses.
- Also known as the Virtual Constructor.



Factory Method Pattern



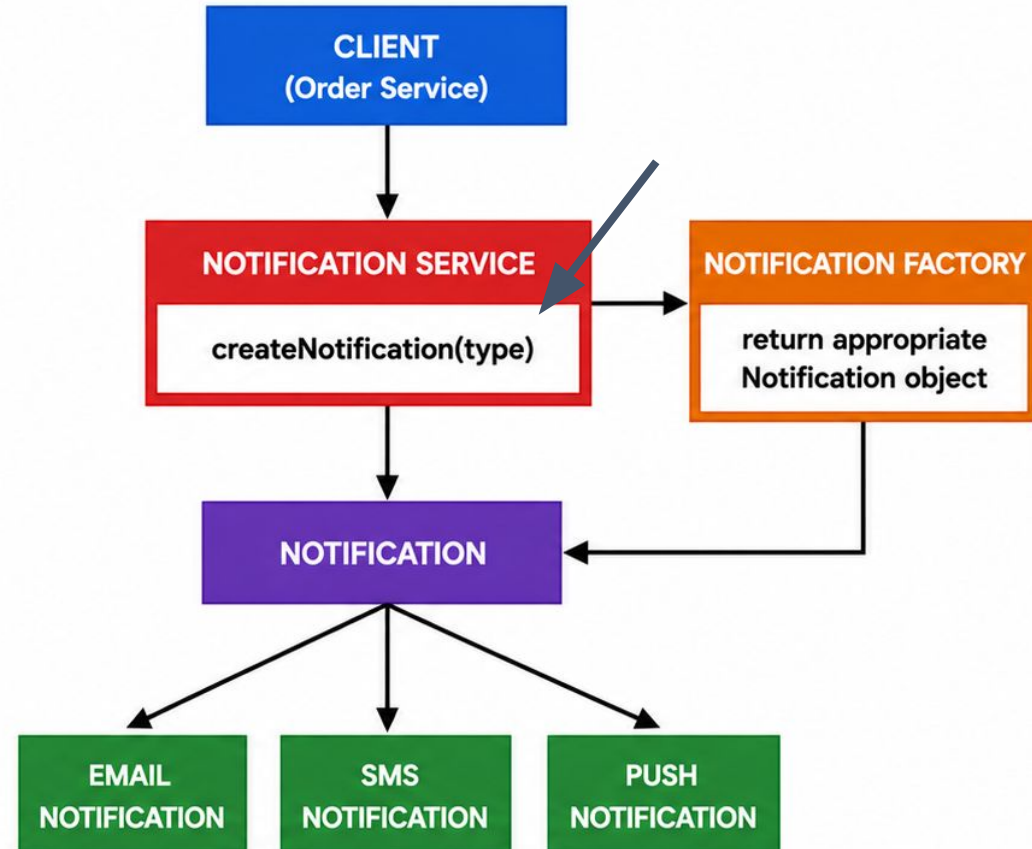
Factory Method Pattern - Example

```
public abstract class Notification {  
    public abstract void open();  
}  
  
public class Email extends Notification {  
    public void send() { /* ... */ }  
}  
  
public class SMS extends Notification {  
    public void send() { /* ... */ }  
}
```

```
public class Application {  
    public void newNotification() {  
        // Notification is abstract - cannot instantiate  
        // Hard-coding a subclass locks us in.  
  
        Notification notification = new Email();  
        notification.send();  
    }  
}
```

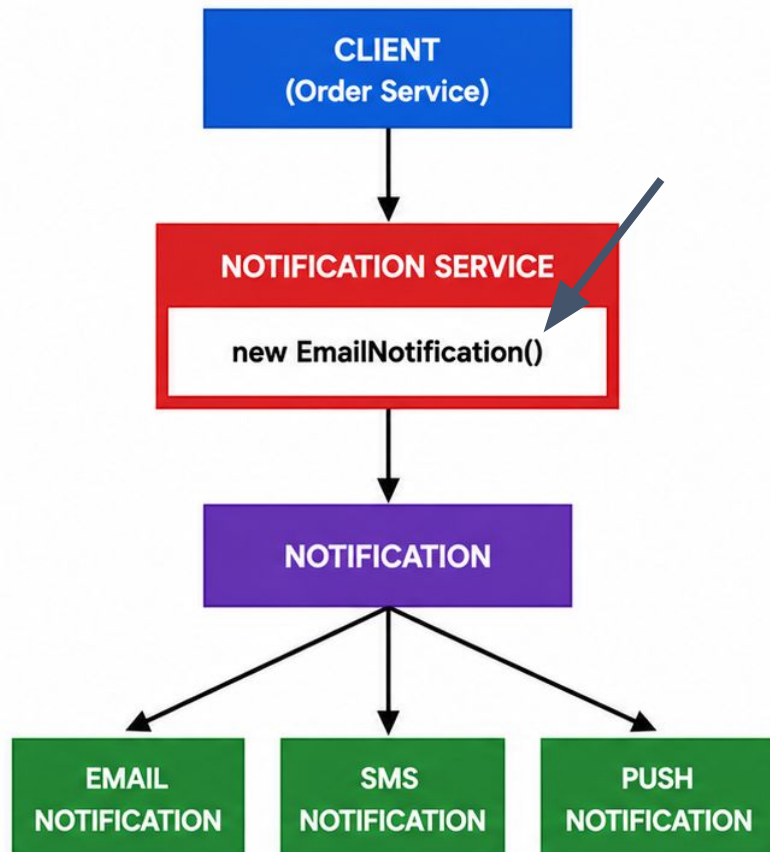
Application knows WHEN a document should be created, not WHAT KIND. Hard-coding the subclass is not a scalable solution.

Factory Method Pattern

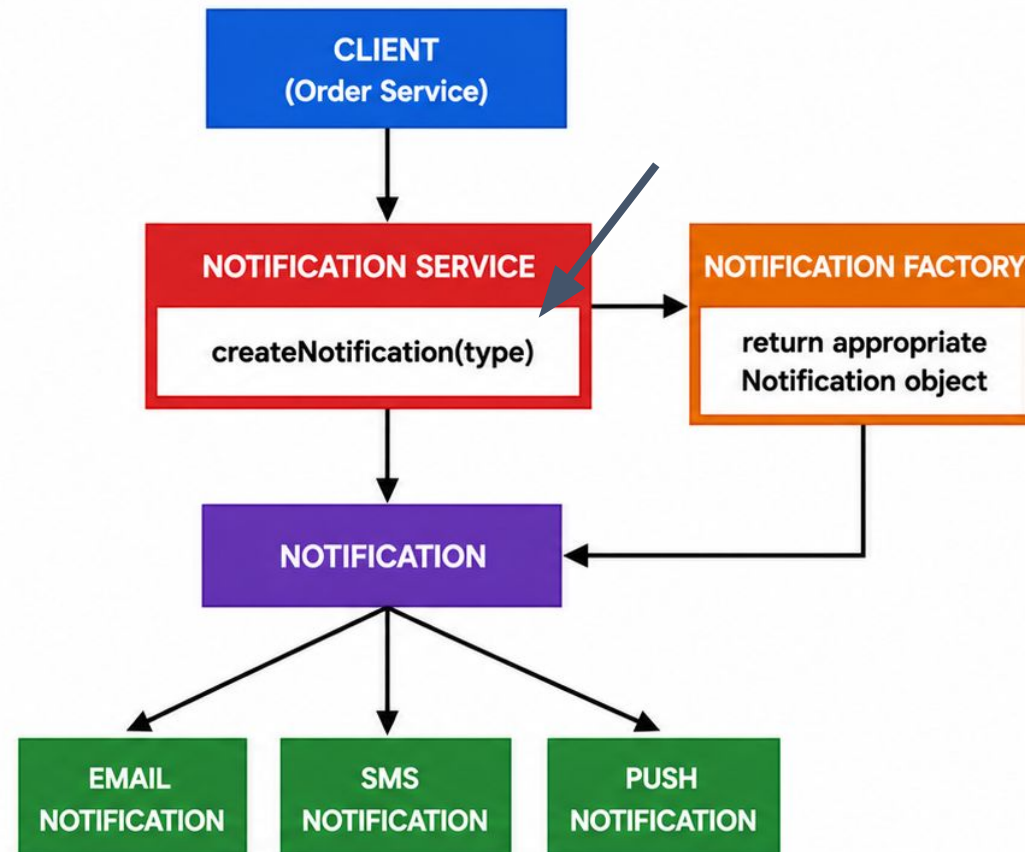


Factory Method Pattern

WITHOUT FACTORY PATTERN

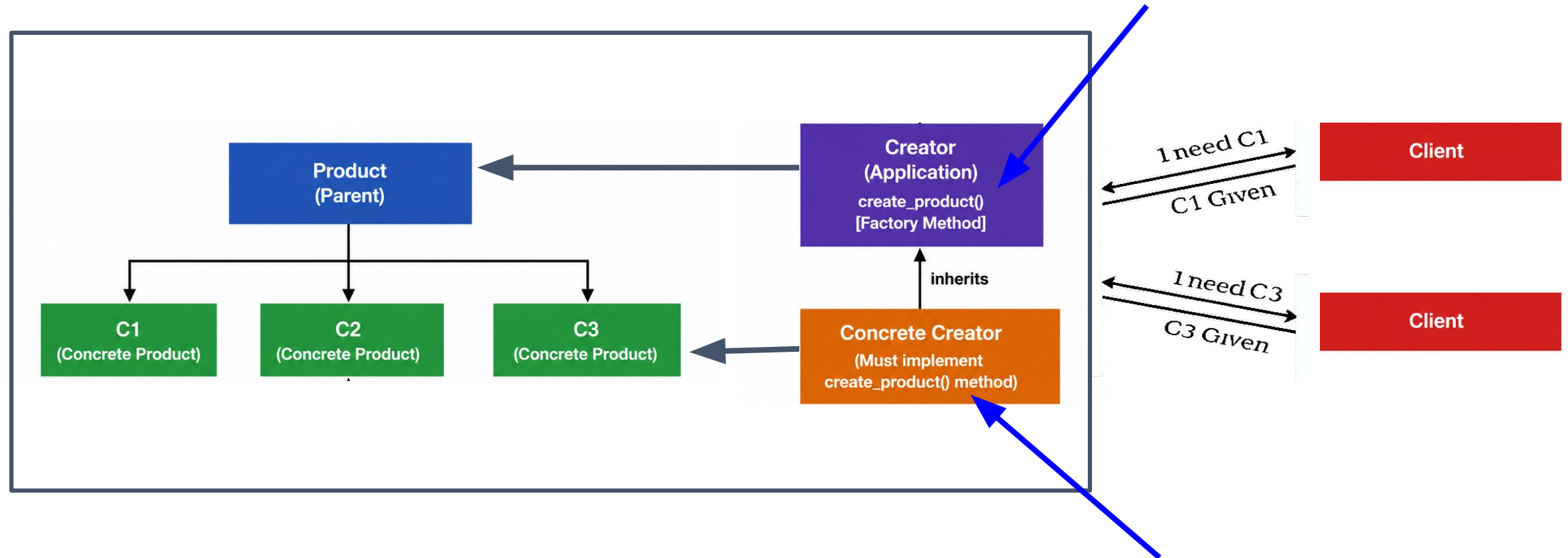


WITH FACTORY PATTERN
(FACTORY METHOD)



Factory Method Pattern

*Abstract Factory method: **create_product()***



Factory Method Pattern

Intent: Define an interface for creating an object, but let subclasses decide which class to instantiate.

Motivation:

- A class cannot anticipate the class of objects it must create (e.g., a framework only knows abstract classes).
- A class wants its subclasses to specify the objects it creates.
- Classes delegate responsibility to one of several helper subclasses, and we want to localize the knowledge of which helper is the delegate.



Factory Method Pattern

Participants:

Product (e.g., Document)

- Defines the interface of the objects the factory method creates.

ConcreteProduct (e.g., MyDocument)

- Implements the Product interface.

Creator (e.g., Application)

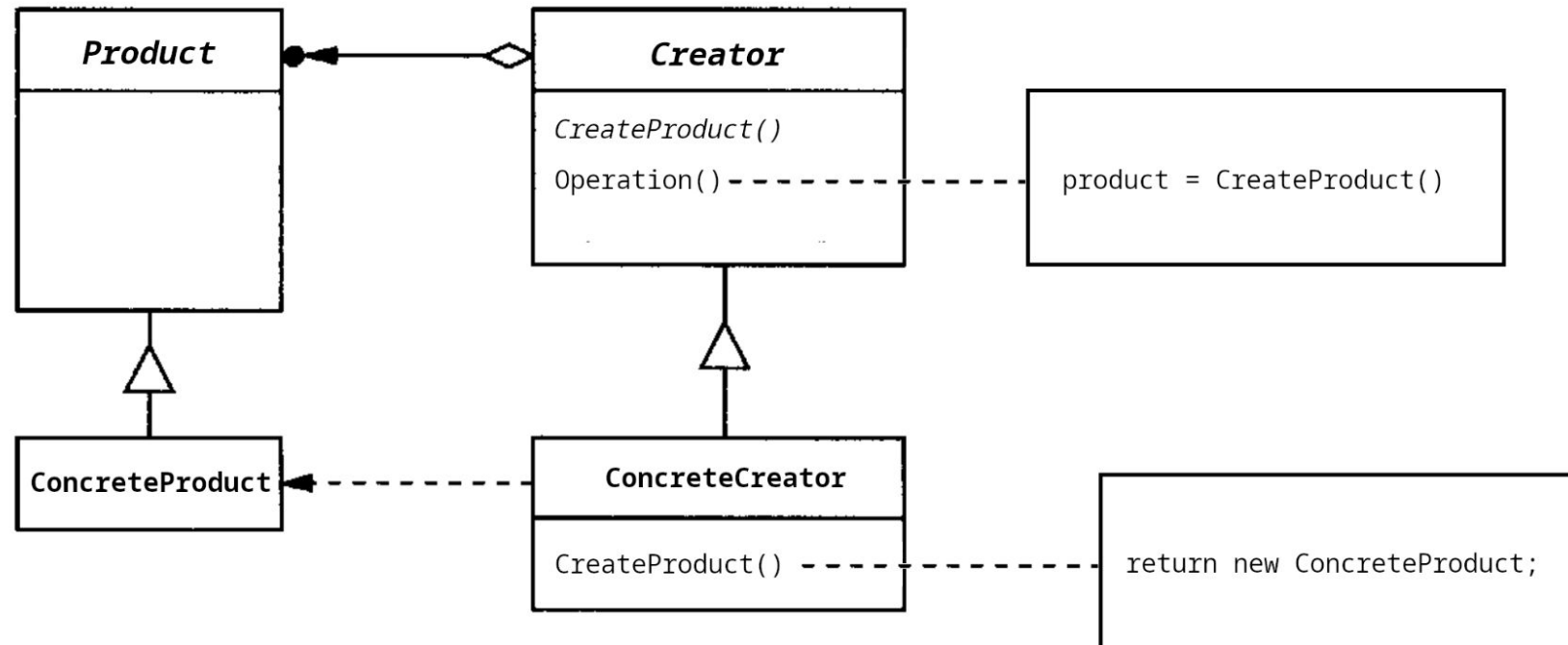
- Declares the factory method, which returns an object of type Product. May call the factory method to create a Product object.

ConcreteCreator (e.g., MyApplication)

- Overrides the factory method to return an instance of a ConcreteProduct.



Factory Method Pattern - Structure



Factory Method Pattern - Structure (Python)

```
from abc import ABC, abstractmethod

class Product(ABC):
    @abstractmethod
    def operation(self): pass

class ConcreteProduct(Product):
    def operation(self):
        print("ConcreteProduct operation")

class Creator(ABC):
    @abstractmethod
    def create_product(self): pass    # the factory method

    def operation(self):
        product = self.create_product()
        product.operation()

class ConcreteCreator(Creator):
    def create_product(self):
        return ConcreteProduct()
```

Factory Method Pattern - Structure (Python)

```
if __name__ == "__main__":  
    # Client code works through the Creator interface;  
    # the concrete subclass decides which Product is built.  
    creator = ConcreteCreator()  
    creator.operation()
```



Factory Method Pattern - Example

```
public abstract class Document {  
    public abstract void open();  
}  
  
public class DrawingDocument extends Document {  
    public void open() { /* ... */ }  
}  
  
public class TextDocument extends Document {  
    public void open() { /* ... */ }  
}
```

```
public class Application {  
    public void newDocument() {  
        // Document is abstract - cannot instantiate  
        // Hard-coding a subclass locks us in.  
        Document doc = new DrawingDocument();  
        doc.open();  
    }  
}  
  
// violates DRY Principle.
```

Application knows WHEN a document should be created, not WHAT KIND. Hard-coding the subclass is not a scalable solution.

Factory Method Pattern - Structure (Python)

OUTPUT=>

ConcreteProduct operation



Factory Method Pattern - Example (Python)

```
from abc import ABC, abstractmethod

class Document(ABC):
    @abstractmethod
    def open(self): pass

class DrawingDocument(Document):
    def open(self): print("Opening drawing document")

class TextDocument(Document):
    def open(self): print("Opening text document")

class Application(ABC):
    @abstractmethod
    def create_document(self): pass    # factory method

    def new_document(self):
        doc = self.create_document()
        doc.open()

class DrawingApplication(Application):
    def create_document(self):
        return DrawingDocument()
```

Factory Method Pattern - Example (Python)

```
if __name__ == "__main__":  
    app = DrawingApplication()  
    app.new_document()  
  
# Application never referenced DrawingDocument directly.  
# Swapping to a TextApplication subclass would change the  
# product with no edits inside Application.new_document.
```



Factory Method Pattern - Structure (Java)

```
public abstract class Product {  
    public abstract void operation();  
}  
  
public class ConcreteProduct extends Product {  
    public void operation() {  
        System.out.println("ConcreteProduct operation");  
    }  
}  
  
public abstract class Creator {  
    public abstract Product createProduct();    // factory method  
  
    public void operation() {  
        Product p = createProduct();  
        p.operation();  
    }  
}  
  
public class ConcreteCreator extends Creator {  
    public Product createProduct() {  
        return new ConcreteProduct();  
    }  
}
```

Factory Method Pattern - Structure (Java)

```
Creator creator = new ConcreteCreator();  
creator.operation();
```



Factory Method Pattern - Example (Java)

```
public abstract class Document {
    public abstract void open();
}

public class DrawingDocument extends Document {
    public void open() { /* ... */ }
}

public abstract class Application {
    public abstract Document createDocument(); // factory method

    public void newDocument() {
        Document doc = createDocument();
        doc.open();
    }
}

public class DrawingApplication extends Application {
    public Document createDocument() {
        return new DrawingDocument();
    }
}
```

Factory Method Pattern - Example (Java)

```
public class Main {  
    public static void main(String[] args) {  
        Application app = new DrawingApplication();  
        app.newDocument();  
    }  
}
```



Thank You

